单位代码：10635

西南大學

本科毕业论文（设计）

本科毕业论文 LaTeX 模版
—附带简单的使用说明

学　　院	数学与统计学院
专　　业	数学与应用数学（师范）
姓　　名	包小敏
学　　号	222016314000000
指导教师	包小敏
答辩日期	2025 年 5 月 8 日

2025 年 5 月

独创声明

本人郑重声明：所呈交的毕业论文（设计），是本人在指导老师的指导下，独立进行研究工作所取得的成果，成果不存在知识产权争议。尽我所知，除文中已经注明引用的内容外，本论文（设计）不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体均已在文中以明确方式标明。

声明的法律后果由本人承担。

作者签名：_____

年　　月　　日

毕业论文（设计）使用授权声明

本人完全了解西南大学关于收集、保存、使用毕业论文（设计）的规定。

本人愿意按照学校要求提交论文（设计）的印刷本和电子版，同意学校保存论文（设计）的印刷本和电子版，或采用影印、数字化或其它复制手段保存论文（设计）；同意学校在不以营利为目的的前提下，建立目录检索与阅览服务系统，公布论文（设计）的部分或全部内容，允许他人依法合理使用。

（保密论文在解密后遵守此规定）

作者签名：_____

年　　月　　日

目 录

独创声明 i

摘要 1

Abstract 1

第 1 章 关于模板的一些说明 2

1.1 模板的构成 2

1.1.1 三级标题 3

1.2 TEX 简介 3

1.3 LATEX 文件的编译 3

第 2 章 与数学有关的一些内容 5

2.1 常见的数学环境示例 5

2.2 矩阵的排版 6

2.3 算法的排版 7

第 3 章 表格、公式与插图 10

3.1 表格 10

3.2 公式 11

3.3 插图 13

第 4 章 参考文献的引用示例 14

第 5 章 程序代码的插入 15

参考文献 16

致谢 17

索引 17

附录 A Matlab 代码 18

附录 B Mathematica 代码 21

附录 C Python 代码 23

附录 D R 代码 **24**

附录 E DES C++ 代码 **25**

本科毕业论文 LaTeX 模版

— 附带简单的使用说明

包小敏

西南大学数学与统计学院，重庆 400715

摘要： 本模版是为西南大学数学与统计学院本科毕业生而设计的 (当然其他学院的学生也可以使用). 模版根据西南大学论文的 Word 模版中所描述的要求编写 [1, 2]. 目的是简化和规范学位论文的撰写，使得论文作者可以将精力集中到论文的内容上而不是浪费在版面设置上. 使用者只需了解 LaTeX 的基本概念即可使用本模版.

关键词： LaTeX; 模版; 论文; 版面; 参数

Undergraduate Thesis LaTeX Template

NAME of Student

School of Mathematics & Statistics，Southwest University，Chongqing 400715

Abstract： This LaTeXtemplate is designed for undergraduate students from the School of Mathematics and Statistics of Southwest University. The template is written according to the requirements described in the Word template of the School of Mathematics and Statistics. The purpose is to simplify and standardize the writing of the thesis, and allows the students to focus their energy on the content of the paper rather than wasting it on formatting. Only basic knowledge of the LaTeXis required to use this template.

Key words： LaTeX; template; thesis; layout; parameters

第 1 章　关于模板的一些说明

本模版是为西南大学数学与统计学院本科毕业生撰写毕业论文而设计的，版面尺寸按照西南大学的论文撰写要求选取 [2]。模板的第一版是 2006 年完成的，经过多年的使用、改进，最终发展成目前的版本。

模板的最近一次修改是 2024 年 8 月。这次修改的变动比较大，首先 西南大学对论文的结构、格式和要求进行了比较大的改动 [1, 2]；其次，为了适应更多的使用者，本次模板的编译采用的是 XeLatex，不是以前的 PdfLatex。

第一次使用本模版时，请先阅读 1.1节；如果对数学公式、符号或定义、定理、证明等各种命令环境不熟悉，请阅读第 2 节；如果对表格或图形的插入不熟悉，请阅读第 3 节；如果对交叉引用不熟悉，请阅读第 4节。如果想了解在 LaTeX 中引入程序代码，如 `Matlab`，`Mathematica`, `Python`，`R` 和 `C/C++` 等的代码，请阅读第 5节, 并分别参考附录 A、B、C、D、E 中的实例.

1.1　模板的构成

本模板由若干文件组成，所有文件都存放在一个主文件夹里。为了使用和管理方便，在主文件夹中又分了七个子文件夹：`preample`，`codes`，`tables`，`declaration`，`zhengwen`，`fujian`，`wenxian`，其中 `preample`，`tables`，`declaration` 中的文件一般不用管。子文件夹 `codes` 中放有一些制作本说明时的附件所用的一些代码文件，如果不需要可以删除。

主文件 mytemplate: 论文的编译一般通过主文件进行。学生的基本信息，如姓名，学号，论文题目等，都在主文件上输入。主文件名可根据自己的需求改动。

文件夹 codes: 文件夹中包含几个程序代码。如果使用者自己没有代码，可以将其删除。

文件夹 zhengwen: 文件夹中包含五个文件，分别对应五章。使用者可以根据自己论文的实际章数来增加或减少这里的文件数，同时录入自己论文相应章节的内容。

文件夹 fujian: 文件夹中包含以下几个文件：

(1) `abstract.tex`，这是论文的摘要。

(2) `thanks.tex`，这是论文的致谢词部分。

(3) `fulu`, 这是论文附录的引入文件。使用者如果没有附录，那就将附录文件 `fulu` 删除。

文件夹 wenxian: 文件夹中包含一个文件，其中是论文中所引用的参考文献。

1.1.1 三级标题

本节为测试三级标题在目录中的显示而设。

1.2 TEX 简介

TEX 是计算机科学家、图灵奖得主 Knuth 教授设计的一款权威的科技论文排版软件！学习 TEX(LATEX)，Knuth 的专著 [6] 无疑是权威之选。该书排版堪称完美，从中可以看出大师的魅力。更重要的是 Knuth 教授无偿公开了 TEX 的所有源代码，也就是说它是开源 (Open Source) 的（通俗地说就是可以免费使用）。正因为这个原因，后人在其基础上开发了 LATEX。LATEX 方便好用，被广泛传播，成了当今世界科技界最权威的论文排版软件。作为一个非常全面的 LATEX 的参考书，[11] 是一个不错的选择。如果阅读英文有困难，作为一个初学者，也可以先看看 [12]。

TEX 和 LATEX 排版软件和 MS 的 Word 软件不同，后者是 “所见即所得”(WYSIWYG，what you see is what you get)，而前者是 “所想即所得”(WYWIWYG，what you want is what you get)。二者在风格上迥然不同，因此学习 LATEX 需要稍微改变一下自己的习惯。

国内较早使用的 LATEX 汉化版是 CTEX，可在 CTEX 网站 免费下载安装。CTEX 安装包自带了一个编辑器 WinEdt, 不过这个编辑器需要注册；否则会频繁跳出弹窗，提示你注册，很是烦人！可以用 TeXstudio 作为编辑器来替代 WinEdt。这时需要将 TeXstudio 和 CTEX 的编译器关联起来，当然也可以选择 TexLive + TeXstudio 的组合。TexLive 可以从清华大学的 镜像网站下载。

1.3 LATEX 文件的编译

LATEX 的编译器有多个，为了避免不必要的麻烦，本模板的编译指定采用 `XeLatex`. 因此用其它编译器编译就会报错. 另外，源文件如果是用其它编辑器录入的, 在用 TeXs-dudio 打开保存之前请确认其是采用的是 UTF-8 汉字编码，否则文件可能会出现乱码. 可以先用记事本将文件打开, 点击 “文件”, 在下拉菜单中选择 “另存为” 后会弹出一个如图 1.1 的窗口，在其下部 “编码” 的下拉窗口中选择 “UTF-8”, 然后点击保存既可转成 UTF-8 的编码.

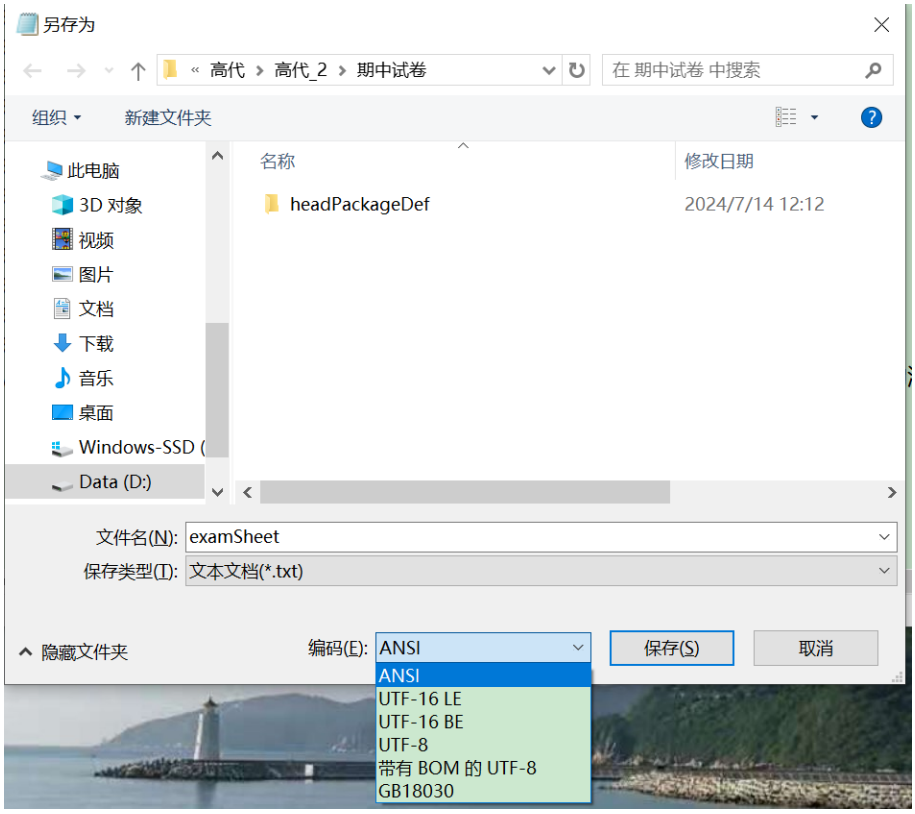

图 1.1　ANSI 编码转 UTF-8 编码

第 2 章　与数学有关的一些内容

2.1　常见的数学环境示例

LaTeX 文件的输入要涉及到一些 LaTeX 命令，尤其是数学符号和公式的输入。这里通过实例来说明如何输入一些与数学有关的内容。建议学生在使用前阅读一遍。通过对比 PDF 文件中的效果和源文件对应的排版命令，初学者可以快速上手使用 LaTeX。

定义 1. 设

$$a_1, a_2, \cdots, a_n, \cdots \tag{2.1}$$

是一个数列。称

$$\sum_{i=1}^{n} a_i$$

为数列 (2.1) 的前 n 项和。

引理 1 (新定理)**.** 若 p 是一个素数，且 p 不能够被 $a \in \mathbb{N}$ 整除，则

$$a^{p-1} \equiv 1 \pmod{p}.$$

证明. 引理的证明在此给出。 □

定理 1 (欧拉定理)**.** 若 $\gcd(a, n) = 1$，则

$$a^{\phi(n)} \equiv 1 \pmod{n}.$$

证明. 下面我们来证明这个定理。 □

引理 2 (费尔马小定理)**.** 若 p 是一个素数，且 p 不能够被 $a \in \mathbb{N}$ 整除，则

$$a^{p-1} \equiv 1 \pmod{p}.$$

证明. 引理的证明在此给出。 □

推论 2. 由组合数的定义立得

$$\binom{n}{k} = \binom{n}{n-k}.$$

性质 1. 设线性变换 $\mathscr{A}$ 在基 $\boldsymbol{\alpha}_1, \boldsymbol{\alpha}_2, \cdots, \boldsymbol{\alpha}_n$ 下矩阵为

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix},$$

向量 $\boldsymbol{\alpha}$ 在基 $\boldsymbol{\alpha}_1, \boldsymbol{\alpha}_2, \cdots, \boldsymbol{\alpha}_n$ 下的坐标为 $(x_1, x_2, \cdots, x_n)$. 则 $\mathscr{A}\boldsymbol{\alpha}$ 在基 $\boldsymbol{\alpha}_1, \boldsymbol{\alpha}_2, \cdots, \boldsymbol{\alpha}_n$ 下的坐标 $(y_1, y_2, \cdots, y_n)$ 可以按公式

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

计算.

例 1 (2010，重庆卷)**.** 例题的内容 $\langle ID, \mu, \gamma \rangle$，$\langle ID, \mu, \gamma \rangle$。

解. 解答的内容。

练习 1. 练习题的内容。

注. 设

$$D = \begin{vmatrix} 5 & 1 & -1 \\ -7 & 1 & 15 \\ 8 & 1 & -21 \end{vmatrix}$$

则 D 的第 3 列元的代数余子式的和是 0。

命题 1. 计算 $\displaystyle\int_0^1 \frac{x}{\sqrt{1-x^2}}\mathrm{d}x$。

解. 因为

$$\int_0^u \frac{x}{\sqrt{1-x^2}}\mathrm{d}x = -\int_0^u \frac{\mathrm{d}(1-x^2)}{2\sqrt{1-x^2}} = -\sqrt{1-x^2}\Big|_0^u = 1-\sqrt{1-u^2},$$

所以 $\displaystyle\int_0^u \frac{x}{\sqrt{1-x^2}}\mathrm{d}x = \lim_{u\to 1^-}\int_0^u \frac{x}{\sqrt{1-x^2}}\mathrm{d}x = 1$。

2.2 矩阵的排版

分块矩阵的排版示例如下：

$$\left[\begin{array}{cc:cc} a_{11} & a_{12} & b_{11} & b_{12} \\ a_{21} & a_{22} & b_{21} & b_{22} \\ \hdashline c_{11} & c_{12} & d_{11} & d_{12} \\ c_{11} & c_{12} & d_{11} & d_{12} \end{array}\right]$$

$$
\mathbf{A} = \begin{array}{c} \\ s_1 \\ s_2 \\ \vdots \\ s_t \end{array} \begin{array}{c} \begin{array}{cccc} n_1 & n_2 & \cdots & n_l \end{array} \\ \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} & \cdots & \mathbf{A}_{1l} \\ \mathbf{A}_{21} & \mathbf{A}_{22} & \cdots & \mathbf{A}_{2l} \\ \vdots & \vdots & & \vdots \\ \mathbf{A}_{t1} & \mathbf{A}_{t2} & \cdots & \mathbf{A}_{tl} \end{pmatrix} \end{array}
$$

2.3 算法的排版

用 LaTeX 可以排出版面非常漂亮的算法。根据所用宏包的不同，所用的命令也有所不同。下面是用宏包 `algorithm` 和 `algorithmic` 进行算法排版的一个例子。所排的算法是著名的欧几里得算法[10] 。

算法 1 Euclidean algorithm (a, b)

输入: 整数 $a, b \neq 0$

输出: 商序列 $q_1, q_2, \cdots, q_m$ 和 $\gcd(a, b)$

```
r_0 ← a
r_1 ← b
m ← 1
while r_m ≠ 0 do
    q_m ← ⌊r_{m-1} / r_m⌋
    r_m ← r_{m-1} − q_m r_m
    m ← m + 1
end while
m ← m − 1
return (q_1, q_2, ··· , q_m; r_m)
comment: r_m = gcd(a, b)
```

算法 2 是另一个标号的。

下面的算法 3 是一个生成 q 进制 n 元反射 Gray 码的算法。

算法 2 Euclidean algorithm (a, b)

输入: 整数 $a, b \neq 0$

输出: 商序列 $q_1, q_2, \cdots, q_m$ 和 $\gcd(a, b)$

$r_0 \leftarrow a$

$r_1 \leftarrow b$

$m \leftarrow 1$

while $r_m \neq 0$ **do**

  $q_m \leftarrow \lfloor \frac{r_{m-1}}{r_m} \rfloor$

  $r_m \leftarrow r_{m-1} - q_m r_m$

  $m \leftarrow m + 1$

end while

$m \leftarrow m - 1$

return $(q_1, q_2, \cdots, q_m; r_m)$

算法 3 Reflected Gray Code Generation (q, n)

输入: 整数 $q > 0, n > 0$

输出: $q^n \times n$ 矩阵 G

$g_{n-1} g_{n-2} \cdots g_0 \longleftarrow 00 \cdots 0$

while $(2 \mid q$ **and** $g_{n-1}, g_{n-2}, \cdots, g_0 \neq q-1, \mathbf{0}_{n-1})$ **or** $(2 \nmid q$ **and** $g_{n-1}, \cdots, g_n \neq (\boldsymbol{q-1})_n$ **do**

  $i \leftarrow 0$

  $\sigma \leftarrow (g_{n-1} + g_{n-2} + \cdots + g_{i+1}) \bmod 2$

  if $\sigma = 0$ **and** $g_i \neq q - 1$ **then**

    $g_i \leftarrow g_i + 1$

  else if $\sigma = 1$ **and** $g_i \neq 0$ **then**

    $g_i \leftarrow g_i - 1$

  end if

  $i \leftarrow i + 1$

  if $i = q - 1$ **then**

    $g_i \leftarrow g_i + 1$

  end if

end while

算法 4 COLLISION-TO-SECOND-PREIMAGE (h)

External: ORACLE-2ND-PREIMAGE

Choose $x \in_R \mathcal{X}$

if ORACLE-2ND-PREIMAGE $(h, x) = x'$ **then**

  return (x, x')

else

  return ("failure")

end if

第 3 章　表格、公式与插图

3.1　表格

盒子测试。表题应写在表格上方正中，表序写在表题左方不加标点，空一格写表题，表题末尾不加标点，全文的表格统一编序，表序必须连续。表题用五号宋体居中，表格内中文用五号宋体，英文用五号 Times New Roman 字体。

表题允许下页接写，接写时表题省略，表头应重复书写，并在右上方写 “续表 xx”。

下面是一些表格的实例。

表题应写在表格上方正中，表序写在表题左方不加标点，空一格写表题，表题末尾不加标点，全文的表格统一编序，表序必须连续。表题用五号宋体居中 (见表 3.1)，表

表 3.1　一些数学符号的 LaTeX 输入命令与效果对照表

输入	输出
`$(\sum x_{ij})^{1/p}$`	$(\sum x_{ij})^{1/p}$
`$\left(\sum x_{ij}\right)^{1/p}$`	$\left(\sum x_{ij}\right)^{1/p}$
`$\bigl(\sum x_{ij}\bigr)^{1/p}$`	$\bigl(\sum x_{ij}\bigr)^{1/p}$
`$\Bigl(\sum x_{ij}\Bigr)^{1/p}$`	$\Bigl(\sum x_{ij}\Bigr)^{1/p}$
`$\biggl(\sum x_{ij}\biggr)^{1/p}$`	$\biggl(\sum x_{ij}\biggr)^{1/p}$
`$\Biggl(\sum x_{ij}\Biggr)^{1/p}$`	$\Biggl(\sum x_{ij}\Biggr)^{1/p}$

格内中文用五号宋体，英文用五号 Times New Roman 字体。

表题允许下页接写，接写时表题省略，表头应重复书写，并在右上方写 “续表 xx”。

下面是一些表格的实例。

表 3.2 三线表示例

姓名	平时成绩	期中考试	期末考试	最后成绩
张三	60	78	76	67
李四	89	92	98	95
王五	95	91	100	97

题号	第一部分					第二部分	总分
	一	二	三	四	五		
得分					√		

左表

a	b	c
d	e	f

右表

w	x	y
z	u	v

3.2 公式

公式应另起一行，正文中的公式、算式或方程式等应编排序号，公式的编号用圆括号括起，序号标注于该式所在行 (当有续行时，应标注于最后一行) 的行末。公式可按章节顺序编号或按全文统一编号。公式序号必须连续，不得重复或跳缺。重复引用的公式不得另编新序号。较长的公式，如必须转行时，最好在等号处转行，如做不到这一点，要在 $+, -, \times, \div$ 等数学符号处转行。数学符号应写在转行处的行首。上下式尽可能在等号 “=” 处对齐。

$$
\begin{aligned}
f(x,y) &= f(0,0) + \frac{1}{1!}\left(x\frac{\partial}{\partial x} + y\frac{\partial}{\partial y}\right) f(0,0) \\
&= \frac{1}{2!}\left(x\frac{\partial}{\partial x} + y\frac{\partial}{\partial y}\right)^2 f(0,0) + \cdots \\
&= \frac{1}{n!}\left(x\frac{\partial}{\partial x} + y\frac{\partial}{\partial y}\right)^n f(0,0) + K
\end{aligned}
$$

$$
\begin{aligned}
f(x,y) &= f(0,0) + \frac{1}{1!}\left(x\frac{\partial}{\partial x} + y\frac{\partial}{\partial y}\right) f(0,0) \\
&= \frac{1}{2!}\left(x\frac{\partial}{\partial x} + y\frac{\partial}{\partial y}\right)^2 f(0,0) + \cdots \\
&= \frac{1}{n!}\left(x\frac{\partial}{\partial x} + y\frac{\partial}{\partial y}\right)^n f(0,0) + K
\end{aligned}
$$

$$
\begin{aligned}
f(x,y) =& f(0,0) + \frac{1}{1!}\left(x\frac{\partial}{\partial x} + y\frac{\partial}{\partial y}\right) f(0,0) \\
&+ \frac{1}{2!}\left(x\frac{\partial}{\partial x} + y\frac{\partial}{\partial y}\right)^2 f(0,0) + \cdots \\
&+ \frac{1}{n!}\left(x\frac{\partial}{\partial x} + y\frac{\partial}{\partial y}\right)^n f(0,0) + K
\end{aligned} \tag{3.1}
$$

$$
\left\{ x \in \mathbf{R} \,\middle|\, 0 < |x| < \frac{5}{3} \right\}
$$

$$
\left\{ x \in \mathbf{R} \Big/ 0 < |x| < \frac{5}{3} \right\}
$$

$$
S = \sum_{i=1}^{n} m_i, \int_a^b f(x)dx = F(x)
$$

$$
\begin{cases} a_{11}x + a_{12}y = b_1 \\ a_{21}x + a_{22}y = b_2 \end{cases}
$$

方程组只要一个编号

$$
\begin{aligned}
a_{11}x + a_{12}y &= b_1 \\
a_{21}x + a_{22}y &= b_2 \\
a_{21}x + a_{22}y &= b_2
\end{aligned} \tag{3.2}
$$

或者

$$
\begin{cases} a_{11}x + a_{12}y = b_1 \\ a_{21}x + a_{22}y = b_2 \end{cases} \tag{3.3}
$$

方程组中如每个方程要单独编号，则可写成

$$
\begin{cases} a_{11}x + a_{12}y = b_1 & (3.4) \\ a_{21}x + a_{22}y = b_2 & (3.5) \end{cases}
$$

(3.4)-(3.5) 或

$$
a_{11}x + a_{12}y = b_1 \tag{3.6}
$$

$$
a_{21}x + a_{22}y = b_2 \tag{3.7}
$$

$$
f(x) = \begin{cases} x+1, & \text{if } x > 1; \\ -x, & \text{if } x \le 1. \end{cases}
$$

$$
a_{11}x + a_{12}y = c_1 \tag{3.8}
$$

$$
a_{21}x + a_{22}y = c_2 \tag{3.9}
$$

3.3 插图

每幅插图应有图序和图题，全文插图可以统一编序，图序必须连续，不得重复或跳缺。图序和图题写在图的下方 (见图 3.1)，五号宋体居中。下面是两种图片并列（图 3.2

图 3.1 数学与统计学院院徽

和图 3.3，图 3.4(a)和图 3.4(b)）的两种情形：

图 3.2 数学与统计学院院徽

图 3.3 西南大学校徽字体

(a) 第一个子图

(b) 第二个子图

图 3.4 西南大学与数学与统计学院

第 4 章　参考文献的引用示例

按论文中参考文献出现的次序，用中括号的数字连续编号，五号宋体，文献顶格，单倍行距 (见例 2)。引用格式按照西南大学学报要求，请注意标点符号。国内高等院校数学专业常用的两本[15, 13]《近世代数》是张禾瑞编著的《近世代数》[13] 和熊全淹编著的《近世代数》[15].

例 2. 参考文献中条目排版示例：

[4] 郑霖，柴宗新，郑远昌等.四川省地理[M].四川科学技术出版社，1994.108-111.

[5] 刘广珠. 高中生考试焦虑成因分析[J].陕西师大学报（哲社版），1995，24（1）：161-164.

这是一个外文参考文献的引用实例[4]。请注意上面两种引用格式，一种是上标，一种是正常。使用者可根据自己的喜好选用。

这是另外一种引用方式 [14]，如果必要也可以这样用。

第 5 章　程序代码的插入

如果论文涉及到较长的计算机程序，可以考虑将其放在附录中。使用下面的命令

```
\lstinputlisting[language=代码语言]{代码文件名}
```

即可将程序代码引入 CTEX 中，这里“代码语言”是指编写代码所使用的语言，如 C/C++，Matlab 等，而“代码文件名”是指存放代码所使用的文件名（如代码文件存放在与 fulu.txt 不同的文件夹中，还需给出那个文件夹的路径）。附录 A—E 给出了 C/C++，Matlab，Python, Mathematica 和 R 的具体例子。

参考文献

[1] 西南大学. 关于印发《西南大学全日制本科学生毕业论文（设计）工作管理办法》的通知. http://jwc.swu.edu.cn/__local/6/D2/C1/4328A82EE22467E1BE3211788BB_70028746_53B64.pdf?e=.pdf

[2] 西南大学. 西校〔*2021*〕*4* 号附件.*docx* http://jwc.swu.edu.cn/info/1020/2347.htm

[3] 西南大学数学与统计学院. 关于 *2017* 届本科毕业论文工作安排的通知.

[4] C.H.Bennett, F.Bessette, G.Brassard, et al, *Experimental quantum cryptography* [J]. Journal of Cryptology, Vol.5, No.1(1992), 3–28.

[5] TEX Guru. *LaTeX 2ε* 用户手册. 1999.

[6] Donald E. Knuth, *The TeXbook* [M]. Addison-Wesley, 1984.

[7] Donald E. Knuth, *The Arts of Computer Programming* [M]. 北京：机械工业出版社, 2008.

[8] 赫尔穆特. 科普卡，帕特里克 W. 达利, *LaTeX* 实用教程（英文版第 4 版）[M]. 北京：机械工业出版社，2005.

[9] 胡伟, *LaTeX 2ε* 完全学习手册（第二版）[M]. 北京：清华大学出版社，2013.4.

[10] D. R. Stinson, *Cryptography—Theory and Practice (Third Edition)*[M]. Chapman & Hall/CRC, Taylor & Francis Group, Roca Raton, FL, USA. 164.

[11] Frank Mittelbach, Michel Goossens. *The LaTeX Companion (Second Edition)*[M]. Addison-Wesley, 2005.

[12] 张林波等. *CCT* 中外文科技排版系统 [M]. 北京：海洋出版社，1993 年。

[13] 张禾瑞, 近世代数 [M]. 高等教育出版社, 1978 年.

[14] Das,M.L., Saxena, A., and Phatak,D.B. *Algorithms and Approaches of Proxy Signature: A Servey*. arXiv:cs/0612098v1 [cs.CR],20 Dec 2006. https://arxiv.org/pdf/cs/0612098.pdf.

[15] 熊全淹, 近世代数 [M]. 武汉大学出版社, 1995 年.

致谢：

本模板目前的版本是由 2006 年完成的初始版经多年反复修改而来。西南大学数学与统计学院前后十八届同学在使用过程中所反馈的意见是本模板发展的强大推动力。在此我要感谢所有使用过模板的同学，尤其是模板完成后最初几年使用过的同学，他们承受了初期模板中存在的问题所带给他们的种种不便，有时甚至可能是痛苦。同时我也要感谢数学与统计学院的领导和某些老师对使用本模板的宽容态度，正是由于他们的宽容，才使得本模板的使用和推广成为可能。我还要特别感谢我的同事彭作祥教授多年来的热情鼓励以及一些有益的建议，他在 2014 年就将模板推荐给统计系 2015 届的同学使用，从而使更多的同学开始接触到 LaTeX。早在 2016 年，学院就“建议所有学生的毕业论文（设计）均使用 latex 模板排版”[3]。到了 2019 年，学院就要求所有毕业生使用 LaTeX 撰写毕业论文，而我本人一直欢迎和鼓励同学们使用我这个模板。

希望大家在使用 LaTeX 编辑完成论文后能体会到当年 D.E.Knuth 教授在修订他的长篇巨著《The Arts of Computer Programming》[7] 时还不得不抽出大量的时间来设计、研制 LaTeX 的前身 TeX 的心情，以及 LaTeX 对当今社会的影响。同时我更希望 LaTeX 能给大家今后的工作、学习和生活带来更多的便利。

由于 LaTeX 一直在不断的更新，同时计算机的操作系统也在不断升级，再加上本人也仅仅是一个业余的 LaTeX 爱好者，水平有限，因此模板难免会有这样或那样的问题。请将问题或者建议 email (xbao@swu.edu.cn) 告诉我。感谢大家的意见和反馈！

包小敏

2024 年 8 月 17 日

于北碚学府小区

附录 A　Matlab 代码

下面是我写的一个 Matlab 代码. 这个代码可以给出下面这个 1989 年第 30 届国际数学奥林匹克（IMO）竞赛的一道试题的一个解答：

求证：集合 $\{1, 2, \cdots, 1989\}$ 可以分为 117 个互不相交的子集

$$A_i, \qquad i = 1, 2, \cdots, 117$$

使得

(1) 每个 A_i 都含有 17 个元素；

(2) 每个 A_i 中所有元素之和相同。

这个代码的文件名为 `equalsumpartition.m`，运行时在 Matlab 命令窗口键入下面的命令

```
equalsumpartition(1989,117)
```

然后回车即可。这里引入所用的命令是

```
\lstinputlisting[language=Matlab]{codes/equalsumpartition.m}
```

注意，因为文件 `equalsumpartition.m` 存放在子文件夹 `codes` 中，所以引入时，文件名前面还必须加上路径名 `codes/`。

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function A = equalsumpartition(l, n)
%EQPARTITON partitions the set {1,2,...,l} into n l/n-subsets, such
   that
% the sums of the elements in any one of the 1/n-subset are equal.
%-------------------------------------------------------------
```

```
% let l = n*m, divide the set {1,2,..., l} into n subsets
% A_1,A_2, ..., A_n, such that
% 1. |A_1| = |A_2| = ... = |A_n| = m;
% 2. sum(A_1) = sum (A_2) = ...= sum(A_n).
% EQPARTITON(l, n) returns an n x m array, the rows of which can be
% used as A_1,A_2, ..., A_n.
%
% Example: if l = 15, n = 5, then
%
%                1   8  15
%                2  10  12
% EQPARTITION = 3   7  14
%                4   9  11
%                5   6  13
%---------------------------
% Author:     Xiaomin Bao
% Created:    June  12, 2007
% Updated:    April 28, 2013
%---------------------------
m = l/n;
a = (1:n)';
b = (n:-1:1)';
A = zeros(n,m);
for i = 1:m
    if mod(i,2) == 1
        A(:,i) = n*(i-1) + a;
    else
        A(:,i) = n*(i-1) + b;
    end
end
if (m > 1) && (mod(m,2) == 1)
    c = zeros(n,1);
    j = 1;
```

```
    for i = n:-2:1
        c(i) = j;
        if i > 1
            c(i-1) = ceil(n/2) + j;
        end
        j = j + 1;
    end
    d = circshift(c,[-1 0]);
    A(:,2) = n + c;
    A(:,3) = 2*n + d;
end
```

附录 B　Mathematica 代码

按第 5 章介绍的方法直接引入 Mathematica 代码目前还有一些问题，一个临时解决办法是先将 Mathematica 代码另存为.txt 文件，然后用命令

```
\lstinputlisting[language=Mathematica]{myfile.txt}
```

引入。下面就是一个按这种方法引入的一个 Mathematica 代码：`primeSieve.txt`，这里用的命令是

```
\lstinputlisting[language=Mathematica]{codes/primeSieve.txt}
```

注意，文件 `primeSieve.txt` 也存放在子文件夹 `codes` 中。这个代码实现了著名的 Eratosthenes 筛法。对一个任意给定的正整数 n，通过 Eratosthenes 筛法，可以找出所有不超过 n 的素数。本代码首先在 $[50, 200]$ 中随机地选一个数作为 n（如果要用其它的数，可通过调整代码中赋给的 `m` 和 `M` 的值来实现），然后给出所有不超过 n 的素数。如果要查看中间结果，只需将一个非 0 的值赋给变量 d，然后运行程序即可。

```
(*----------------------------------*)
(*File name:        primeSieve.nb    *)
(*author:           Xiaomin Bao      *)
(*Created on:       August 9, 2006   *)
(*Last revized on:  June 12, 2015    *)
(*----------------------------------*)
(* This code implements the sieve of Eratosthenes *)
primeSieve[n_,d_]:=Module[{u=Range[n], p=2},
    If[d != 0,Print["Initially, the set of numbers = ", u]];
    While[p<=Sqrt[n],
        Do[u[[i]]= 0, {i,p^2,n,p}];(*replace the numbers of the form
            tpp with 0, where 1 <= t *)
```

```
        If[d != 0,Print[ "when p = ",  p, ",  numbers remained = ",
           u]];
        p++;
        While[p<=Sqrt[n]&&u[[p]]==0, p++]; (* if u[[p]] = 0, then
           check next number *)
    ];
    Select[u,(#>1)&] (* excluding all components which equal to 0 *)
]
m = 50;
M = 200; (* If you want to test a specific number, just set m and M
   to that number *)
n = RandomInteger[{m,M}];(* Select a number randomly from the
   interval [m,M]*)
d = 0; (* If d != 0, the code will display all intermidiate results*)
t = primeSieve[n,d];
Print["The set of primes that are less than or equal to ", n, " = ",
   t];
```

附录 C　Python 代码

下面是一个文件名为 `test.py` 的 Python 代码，采用的是和 Mathematica 代码相同的引入命令.

```
\lstinputlisting[language=Python]{codes/test.txt}
```

```python
# -*- coding: UTF-8 -*-
# Filename  : test.py
# authorby  : www.runoob.com
# 随机生成 0 ~ 9 之间的一个数
# 导入 random (随机数) 模块
import random
print(random.ranint(0,9))
```

附录 D　R 代码

此例子由彭作祥教授提供，是 **R** 软件下编写的程序，为寿险精算中的一个简单例子。计算在每年末偿还金额中的利息和本金金额。这个代码的文件名为 example.R，在 **R** 中打开运行即可。引入命令是

```
\lstinputlisting[language=R]{codes/example.R}
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
# 等额分期偿贷
B0 <- 20000; n <- 5; i <- 0.06; v <- 1/(1+i);
a <-function(k)
{a <-(1-v^k)/i
}
R <- B0/a(n);
#每年末偿还金额中的利息和本金金额 # 利息
Ik <-array(0, c(1, n, 1))
for(k in 1:n)
{
  Ik[k] = i*R*a(n-k+1)
}
Ik <-round(Ik, digits = 2); Ik # 本金
Pk <-array(0, c(1, n,1))
for (k in 1:n)
{
  Pk[k] = B0/a(n)*v^(n-k+1)
}
Pk <- round(Pk, digits = 2);
Pk
```

附录 E　DES C++ 代码

下面是一个文件名为 mytestdes.cpp 的 C++ 代码，引入命令是

```
\lstinputlisting[language={[ANSI]C++}]{codes/mytestdes.cpp}
```

这个代码实现了著名的对称密码算法 DES。

```
/*-----------------------------------------------------------------
 * File name:    mytestdes.cpp
 *
 * Author:       Xiaomin Bao
 * Last updated: Feb. 22, 2001
 *-----------------------------------------------------------------*/
/*
  This code implements the encryption of DES, it reads in a key and a
  plaintext (both in hexdecimal form) from files, then prints out the
  ciphertext (binary form).
*/

#include <iostream>
#include <fstream>
#include <vector>
#include <iomanip>

using namespace std;

typedef vector<int> binVector;

//-----------------------------------------------------------------
```

```
binVector KeyIP(binVector);
binVector C_D_Finder(binVector, int);
void Print_List(binVector);
binVector K_Finder(binVector,int);
binVector IP(binVector);
binVector List_Extender(binVector);
binVector Lists_Plus(binVector, binVector);
binVector Get_Half_L(binVector, int);
binVector Func_f(binVector);
binVector R_Finder(binVector, binVector, binVector, int);
binVector Lists_Merger(binVector, binVector);
binVector InvIP(binVector);

// external function, the names of the file which contains it is
// hexToBinary.cpp

binVector hexToBinary(char *);

// external function, name of the file: binaryToHex.cpp

vector <char> binaryToHex(binVector);
//-------------------------------------------------------------------

int main(){

   char *keyInput  = new char[16];
   char *textInput = new char[16];

   binVector Key, plaintext;
   vector <char> ciphertext; // ciphertext in hexdecimal form

   binVector  c[17], d[17], k[16], L[17], R[17];
```

```
ifstream keyIn, PlainTextIn;

keyIn.open("keyHexForm.txt");
if(!keyIn){
   cout << "Can not open the file." << endl;
   return 1;
}

PlainTextIn.open("plaintextHexForm.txt");
if(!PlainTextIn){
   cout << "Can not open the plaintext file." << endl;
   return 1;
}

keyIn.read(keyInput, 16); // read in key

Key = hexToBinary(keyInput);
cout << "\n   The key (binary form) is:\n";
Print_List(Key);

PlainTextIn.read(textInput,16); // read in plaintext

plaintext = hexToBinary(textInput);

cout << "   The plaintext (binary form) is:\n";
Print_List(plaintext);

c[0]=Get_Half_L(KeyIP(Key),1);
d[0]=Get_Half_L(KeyIP(Key),29);

c[1]=C_D_Finder(c[0],1);
d[1]=C_D_Finder(d[0],1);
```

```
c[2]=C_D_Finder(c[1],1);
d[2]=C_D_Finder(d[1],1);

for(int i = 3; i <= 8; i++){
   c[i] = C_D_Finder(c[i-1],2);
   d[i] = C_D_Finder(d[i-1],2);
}

c[9]=C_D_Finder(c[8],1);
d[9]=C_D_Finder(d[8],1);

for(int i =10; i <= 15; i++){
   c[i] = C_D_Finder(c[i-1],2);
   d[i] = C_D_Finder(d[i-1],2);
}

c[16]=C_D_Finder(c[15],1);
d[16]=C_D_Finder(d[15],1);

for(int i = 0; i < 16; i++)
   k[i] = K_Finder(Lists_Merger(c[i+1],d[i+1]),i+1);

L[0]=Get_Half_L(IP(plaintext),1);
R[0]=Get_Half_L(IP(plaintext),33);

for(int i = 1; i <= 16; i++){
  L[i] = R[i-1];
  R[i] = R_Finder(L[i-1],R[i-1],k[i-1],i);
}

cout << "   The cryptotext is:\n";
Print_List(InvIP(Lists_Merger(R[16],L[16])));
```

```
   ciphertext = binaryToHex(InvIP(Lists_Merger(R[16],L[16])));

   for(int i=0; i < ciphertext.size(); i++)
      ciphertext[i];
   cout << endl;
   keyIn.close();
   PlainTextIn.close();

   return 0;
}//main

//--------------KeyIP-----------------------------------------------
binVector KeyIP(binVector I){ // permute key
   binVector tmp;

   for(int i=1; i<=3; i++){
      for(int j=0; j<8; j++)
        tmp.push_back(I[55+i-8*j]);
   }
   for(int i=1;i<=4;i++)
      tmp.push_back(I[59-8*(i-1)]);


   for(int i=1; i<=3; i++){
      for(int j=0; j<8; j++)
        tmp.push_back(I[63-i-8*j]);
   }
   for(int i=1;i<=4;i++)
       tmp.push_back(I[27-8*(i-1)]);

   return tmp;
}//KeyIP
```

```
//--------------------C_D_Finder-----------------------------------
binVector C_D_Finder(binVector L, int shift){// find ci and di
   binVector tmp;

   for(int i=shift; i<L.size(); i++)
      tmp.push_back(L[i]);
   for(int j=0; j<shift; j++)
      tmp.push_back(L[j]);
   return tmp;
}//C_D_Finder

//-----------------------Print_List---------------------------------
void Print_List(binVector L){//print binary string, 8 bits per row
   cout <<"   ----------------------------------------------\n\n";
   for(int i=1; i <= L.size(); i++){
      cout << setw(6) << L[i-1];
      if(i%8==0)
         cout << endl;
   }
   cout << endl;
}//Print_List

//----------------K_Finder------------------------------------------
binVector K_Finder(binVector C, int n){// find kn
   binVector tmp;

   tmp.push_back(C[13]);
   tmp.push_back(C[16]);
   tmp.push_back(C[10]);
   tmp.push_back(C[23]);
   tmp.push_back(C[0]);
   tmp.push_back(C[4]);
```

```
tmp.push_back(C[2]);
tmp.push_back(C[27]);
tmp.push_back(C[14]);
tmp.push_back(C[5]);
tmp.push_back(C[20]);
tmp.push_back(C[9]);

tmp.push_back(C[22]);
tmp.push_back(C[18]);
tmp.push_back(C[11]);
tmp.push_back(C[3]);
tmp.push_back(C[25]);
tmp.push_back(C[7]);

tmp.push_back(C[15]);
tmp.push_back(C[6]);
tmp.push_back(C[26]);
tmp.push_back(C[19]);
tmp.push_back(C[12]);
tmp.push_back(C[1]);

tmp.push_back(C[40]);
tmp.push_back(C[51]);
tmp.push_back(C[30]);
tmp.push_back(C[36]);
tmp.push_back(C[46]);
tmp.push_back(C[54]);

tmp.push_back(C[29]);
tmp.push_back(C[39]);
tmp.push_back(C[50]);
tmp.push_back(C[44]);
tmp.push_back(C[32]);
```

```
    tmp.push_back(C[47]);

    tmp.push_back(C[43]);
    tmp.push_back(C[48]);
    tmp.push_back(C[38]);
    tmp.push_back(C[55]);
    tmp.push_back(C[33]);
    tmp.push_back(C[52]);

    tmp.push_back(C[45]);
    tmp.push_back(C[41]);
    tmp.push_back(C[49]);
    tmp.push_back(C[35]);
    tmp.push_back(C[28]);
    tmp.push_back(C[31]);

    //cout << "\nk" << n << " is:\n";
    //Print_List(tmp);

    return tmp;
}//K_Finder

//---------------IP-----------------------------------------------------

binVector IP(binVector P){//find L0 and R0
    binVector tmp;

    for(int i=1;i<=4;i++){
        for(int j=1;j<=8;j++)
            tmp.push_back(P[57+2*(i-1)-8*(j-1)]);
    }
    for(int i=1;i<=4;i++){
        for(int j=1;j<=8;j++)
```

```
        tmp.push_back(P[56+2*(i-1)-8*(j-1)]);
    }
    return tmp;
}//IP

//-------------------------List_Extender-----------------------------
binVector List_Extender(binVector R){// Extend R from 32 bits to 48
    bits
    binVector tmp;
    tmp.push_back(R[31]);

    for(int i=1;i<=5;i++)
        tmp.push_back(R[i-1]);

    for(int j=1;j<=7;j++){
        for(int l=0; l<6;l++)
            tmp.push_back(R[(3+4*(j-1)+l)%32]);
    }
    return tmp;
}//List_Extender

//----------------------Lists_Plus----------------------------------

binVector Lists_Plus(binVector L, binVector R){//mod 2 sum of L and R
    binVector tmp;
    if(L.size()!=R.size())
        cout << " The list sizes are not equal.\n";
    for(int i=1; i<=R.size(); i++)
        tmp.push_back((L[i-1]+R[i-1])%2);
    return tmp;
}//Lists_Plus

//---------------------Get_Half_L----------------------------------
```

```
binVector Get_Half_L(binVector  long_list, int n){// get half part
   of a
   binVector tmp;                                   // list. If n=1,
      then
   for(int i=n; i<n+long_list.size()/2; i++)        // first part;if
      tmp.push_back(long_list[i-1]);                // n=(size()/2)+1,
   return tmp;                                      // then last part.
}//Get_Half_L

//-------------------Func_f--------------------------------------
binVector Func_f(binVector input){// perform the last permutation of
   f
   int data[8][4][16] = {
14,  4, 13,  1,  2, 15, 11,  8,  3, 10,  6, 12,  5,  9,  0,  7,
 0, 15,  7,  4, 14,  2, 13,  1, 10,  6, 12, 11,  9,  5,  3,  8,
 4,  1, 14,  8, 13,  6,  2, 11, 15, 12,  9,  7,  3, 10,  5,  0,
15, 12,  8,  2,  4,  9,  1,  7,  5, 11,  3, 14, 10,  0,  6, 13,

15,  1,  8, 14,  6, 11,  3,  4,  9,  7,  2, 13, 12,  0,  5, 10,
 3, 13,  4,  7, 15,  2,  8, 14, 12,  0,  1, 10,  6,  9, 11,  5,
 0, 14,  7, 11, 10,  4, 13,  1,  5,  8, 12,  6,  9,  3,  2, 15,
13,  8, 10,  1,  3, 15,  4,  2, 11,  6,  7, 12,  0,  5, 14,  9,

10,  0,  9, 14,  6,  3, 15,  5,  1, 13, 12,  7, 11,  4,  2,  8,
13,  7,  0,  9,  3,  4,  6, 10,  2,  8,  5, 14, 12, 11, 15,  1,
13,  6,  4,  9,  8, 15,  3,  0, 11,  1,  2, 12,  5, 10, 14,  7,
 1, 10, 13,  0,  6,  9,  8,  7,  4, 15, 14,  3, 11,  5,  2, 12,

 7, 13, 14,  3,  0,  6,  9, 10,  1,  2,  8,  5, 11, 12,  4, 15,
13,  8, 11,  5,  6, 15,  0,  3,  4,  7,  2, 12,  1, 10, 14,  9,
10,  6,  9,  0, 12, 11,  7, 13, 15,  1,  3, 14,  5,  2,  8,  4,
 3, 15,  0,  6, 10,  1, 13,  8,  9,  4,  5, 11, 12,  7,  2, 14,
```

```
 2, 12,  4,  1,  7, 10, 11,  6,  8,  5,  3, 15, 13,  0, 14,  9,
14, 11,  2, 12,  4,  7, 13,  1,  5,  0, 15, 10,  3,  9,  8,  6,
 4,  2,  1, 11, 10, 13,  7,  8, 15,  9, 12,  5,  6,  3,  0, 14,
11,  8, 12,  7,  1, 14,  2, 13,  6, 15,  0,  9, 10,  4,  5,  3,

12,  1, 10, 15,  9,  2,  6,  8,  0, 13,  3,  4, 14,  7,  5, 11,
10, 15,  4,  2,  7, 12,  9,  5,  6,  1, 13, 14,  0, 11,  3,  8,
 9, 14, 15,  5,  2,  8, 12,  3,  7,  0,  4, 10,  1, 13, 11,  6,
 4,  3,  2, 12,  9,  5, 15, 10, 11, 14,  1,  7,  6,  0,  8, 13,

 4, 11,  2, 14, 15,  0,  8, 13,  3, 12,  9,  7,  5, 10,  6,  1,
13,  0, 11,  7,  4,  9,  1, 10, 14,  3,  5, 12,  2, 15,  8,  6,
 1,  4, 11, 13, 12,  3,  7, 14, 10, 15,  6,  8,  0,  5,  9,  2,
 6, 11, 13,  8,  1,  4, 10,  7,  9,  5,  0, 15, 14,  2,  3, 12,

13,  2,  8,  4,  6, 15, 11,  1, 10,  9,  3, 14,  5,  0, 12,  7,
 1, 15, 13,  8, 10,  3,  7,  4, 12,  5,  6, 11,  0, 14,  9,  2,
 7, 11,  4,  1,  9, 12, 14,  2,  0,  6, 10, 13, 15,  3,  5,  8,
 2,  1, 14,  7,  4, 10,  8, 13, 15, 12,  9,  0,  3,  5,  6, 11 };

    int  p[8][6];
    int  bin[48],num[8];
    binVector output;
    int n, m;

    for( n = 0; n< 8; n++){
        for( m = 0; m<6; m++){
            p[n][m] = input[n*6+m];
        }
    }

    for( n = 0; n < 8; n++){
        int row, column;
```

```
    row = p[n][0]*2 + p[n][5];
    column= p[n][1]*8 + p[n][2]*4 + p[n][3]*2 +p[n][4];
    num[n] = data[n][row][column];
}

for( n = 0; n < 8; n++){
    switch (num[n]){
    case 0 :
            bin[n*4]   = 0;
            bin[n*4+1] = 0;
            bin[n*4+2] = 0;
            bin[n*4+3] = 0;
            break;

    case 1 :
            bin[n*4]   = 0;
            bin[n*4+1] = 0;
            bin[n*4+2] = 0;
            bin[n*4+3] = 1;
            break;

    case 2 :
            bin[n*4]   = 0;
            bin[n*4+1] = 0;
            bin[n*4+2] = 1;
            bin[n*4+3] = 0;
            break;

    case 3 :
            bin[n*4]   = 0;
            bin[n*4+1] = 0;
            bin[n*4+2] = 1;
            bin[n*4+3] = 1;
```

```
        break;

case 4 :
        bin[n*4]   = 0;
        bin[n*4+1] = 1;
        bin[n*4+2] = 0;
        bin[n*4+3] = 0;
        break;

case 5 :
        bin[n*4]   = 0;
        bin[n*4+1] = 1;
        bin[n*4+2] = 0;
        bin[n*4+3] = 1;
        break;

case 6 :
        bin[n*4]   = 0;
        bin[n*4+1] = 1;
        bin[n*4+2] = 1;
        bin[n*4+3] = 0;
        break;

case 7 :
        bin[n*4]   = 0;
        bin[n*4+1] = 1;
        bin[n*4+2] = 1;
        bin[n*4+3] = 1;
        break;

case 8 :
        bin[n*4]   = 1;
        bin[n*4+1] = 0;
```

```
        bin[n*4+2] = 0;
        bin[n*4+3] = 0;
        break;

case 9 :
        bin[n*4]   = 1;
        bin[n*4+1] = 0;
        bin[n*4+2] = 0;
        bin[n*4+3] = 1;
        break;

case 10 :
        bin[n*4]   = 1;
        bin[n*4+1] = 0;
        bin[n*4+2] = 1;
        bin[n*4+3] = 0;
        break;

case 11 :
        bin[n*4]   = 1;
        bin[n*4+1] = 0;
        bin[n*4+2] = 1;
        bin[n*4+3] = 1;
        break;

case 12 :
        bin[n*4]   = 1;
        bin[n*4+1] = 1;
        bin[n*4+2] = 0;
        bin[n*4+3] = 0;
        break;

case 13 :
```

```
            bin[n*4]   = 1;
            bin[n*4+1] = 1;
            bin[n*4+2] = 0;
            bin[n*4+3] = 1;
            break;

    case 14 :
            bin[n*4]   = 1;
            bin[n*4+1] = 1;
            bin[n*4+2] = 1;
            bin[n*4+3] = 0;
            break;

    case 15 :
            bin[n*4]   = 1;
            bin[n*4+1] = 1;
            bin[n*4+2] = 1;
            bin[n*4+3] = 1;
            break;

    default :
            cout <<"Error detected!\n\n";

    }

}

output.push_back(bin[15]);
output.push_back(bin[6]);
output.push_back(bin[19]);
output.push_back(bin[20]);

output.push_back(bin[28]);
```

```
output.push_back(bin[11]);
output.push_back(bin[27]);
output.push_back(bin[16]);

output.push_back(bin[0]);
output.push_back(bin[14]);
output.push_back(bin[22]);
output.push_back(bin[25]);

output.push_back(bin[4]);
output.push_back(bin[17]);
output.push_back(bin[30]);
output.push_back(bin[9]);

output.push_back(bin[1]);
output.push_back(bin[7]);
output.push_back(bin[23]);
output.push_back(bin[13]);

output.push_back(bin[31]);
output.push_back(bin[26]);
output.push_back(bin[2]);
output.push_back(bin[8]);

output.push_back(bin[18]);
output.push_back(bin[12]);
output.push_back(bin[29]);
output.push_back(bin[5]);

output.push_back(bin[21]);
output.push_back(bin[10]);
output.push_back(bin[3]);
output.push_back(bin[24]);
```

```
    return output;
}//Func_f

//---------------------R_Finder-------------------------------------
binVector R_Finder(binVector L, binVector R, binVector k, int n){
   binVector tmp;
   tmp=Lists_Plus(L,Func_f(Lists_Plus(List_Extender(R),k)));

   //cout << "     R" << n << " is:\n";

   //Print_List(tmp);

   return tmp;
}//R_Finder

//-------------------Lists_Merger-----------------------------------
binVector Lists_Merger(binVector F, binVector L){// append L to F
   binVector tmp;
      for(int i=1; i<=F.size(); i++)
         tmp.push_back(F[i-1]);
      for(int j=1; j<=L.size(); j++)
         tmp.push_back(L[j-1]);
   return tmp;
}//Lists_Merger

//--------------------InvIP-----------------------------------------
binVector InvIP(binVector C){// perform inverse permutation of IP
   binVector tmp;
   for(int i=1; i<=8; i++){
      tmp.push_back(C[40-i]);
      tmp.push_back(C[8-i]);
      tmp.push_back(C[48-i]);
      tmp.push_back(C[16-i]);
```

```
        tmp.push_back(C[56-i]);
        tmp.push_back(C[24-i]);
        tmp.push_back(C[64-i]);
        tmp.push_back(C[32-i]);
    }
    return tmp;
}//InvIP

//=====================================================================
```